Table of Contents

1.Normal ECG Interval Ranges 2.Voltage Ranges (Typical Reference Values) 3.ECG Batch Analysis with Interval Detection, Plotting & Diagnosis

1.Normal ECG Interval Ranges

Interval	Description	Normal Range
PR interval	Time from atrial to ventricular depolarization	**120–200 ms**
QRS duration	Ventricular depolarization	**< 120 ms**
QT interval	Ventricular depolarization + repolarization	**< 440–460 ms** (varies by sex and HR)
RR interval	Time between R-peaks (1 cardiac cycle)	**600–1000 ms** (HR 60–100 bpm)
Heart Rate (HR)	Beats per minute	**60–100 bpm**
QTc (Corrected QT)	QT interval corrected for HR using Bazett's formula	**< 440 ms (men), < 460 ms (women)**

- **PR interval > 200 ms** → Possible **1st-degree AV block**
- **QRS > 120 ms** → Possible **bundle branch block or ventricular rhythm**
- **QTc > 450–460 ms** → Risk of **ventricular arrhythmia** (e.g., **Torsades de Pointes**)
- **RR interval** is inversely related to heart rate:
- 1000 ms → 60 bpm
- 750 ms → 80 bpm
- 600 ms → 100 bpm

2.Voltage Ranges (Typical Reference Values)

Component	Normal Amplitude
P wave	0.1 – 0.25 mV
QRS	0.5 – 2.5 mV (R peak)
T wave	0.1 – 0.5 mV

3.ECG Batch Analysis with Interval Detection, Plotting & Diagnosis

```
% List of ECG files
```

```
 file_list = {'test_data1.csv', 'test_data2.csv', 'test_data3.csv', 'test_data4.csv',
'test_data5.csv', 'test_data6.csv', 'test_data7.csv', 'test_data8.csv', 'test_data9.csv',
'test_data10.csv', 'test_data11.csv'};

 for person_id = 1:length(file_list)
    filename = file_list{person_id};

    % Load ECG data
    data = readmatrix(filename);
    time = data(:,1);
```

```
    voltage = data(:,2);

    % Estimate sampling frequency
    fs = 1 / mean(diff(time));

    % Butterworth low-pass filter
    [b, a] = butter(4, 20/(fs/2), 'low');
    filtered_ecg = filtfilt(b, a, voltage);

    % Detect R-peaks
    N = length(filtered_ecg);
    min_peak_dist = min(round(0.6 * fs), floor(0.5 * N));
    [~, locs_R] = findpeaks(filtered_ecg, 'MinPeakHeight', max(filtered_ecg)*0.6,
'MinPeakDistance', min_peak_dist);

    % Initialize wave positions
    P_locs = []; Q_locs = []; S_locs = []; T_locs = [];

    for i = 1:length(locs_R)
        r = locs_R(i);

        % Q wave
        q_start = max(1, r - round(0.04*fs));
        [~, q_rel] = min(filtered_ecg(q_start:r));
        Q_locs(end+1) = q_start + q_rel - 1;

        % S wave
        s_end = min(length(filtered_ecg), r + round(0.04*fs));
        [~, s_rel] = min(filtered_ecg(r:s_end));
        S_locs(end+1) = r + s_rel - 1;

        % P wave
        p_start = max(1, Q_locs(end) - round(0.15*fs));
        [~, p_rel] = max(filtered_ecg(p_start:Q_locs(end)));
        P_locs(end+1) = p_start + p_rel - 1;

        % T wave
        t_end = min(length(filtered_ecg), S_locs(end) + round(0.3*fs));
        [~, t_rel] = max(filtered_ecg(S_locs(end):t_end));
        T_locs(end+1) = S_locs(end) + t_rel - 1;
    end

    % Interval Calculations
    num_beats = length(locs_R) - 1;

    if num_beats < 1
        fprintf('Person %d: Not enough R-peaks for interval analysis.\n', person_id);
        continue;
    end

    PR_intervals = zeros(1, num_beats);
```

```
QRS_intervals = zeros(1, num_beats);
QT_intervals = zeros(1, num_beats);
RR_intervals = zeros(1, num_beats);

for i = 1:num_beats
    PR_intervals(i) = time(locs_R(i)) - time(P_locs(i));
    QRS_intervals(i) = time(S_locs(i)) - time(Q_locs(i));
    QT_intervals(i) = time(T_locs(i)) - time(Q_locs(i));
    RR_intervals(i) = time(locs_R(i+1)) - time(locs_R(i));
end

% Convert to milliseconds
avg_PR = mean(PR_intervals) * 1000;
avg_QRS = mean(QRS_intervals) * 1000;
avg_QT = mean(QT_intervals) * 1000;
avg_RR = mean(RR_intervals) * 1000;
heart_rate = 60000 / avg_RR;
QTc = avg_QT / sqrt(avg_RR / 1000);

% Simple rule-based diagnosis
has_PR_abnormal = avg_PR < 120 || avg_PR > 200;
has_QRS_abnormal = avg_QRS > 120;
has_QT_abnormal = QTc > 450;
has_RR_abnormal = heart_rate < 60 || heart_rate > 100;

if has_PR_abnormal || has_QRS_abnormal || has_QT_abnormal || has_RR_abnormal
    disease_flag = 'YES';
else
    disease_flag = 'NO';
end

 % Plot ECG with PQRST markers
figure('Name', sprintf('Person %d - ECG with PQRST', person_id));
plot(time, filtered_ecg, 'b'); hold on;
plot(time(P_locs), filtered_ecg(P_locs), 'go', 'DisplayName', 'P wave');
plot(time(Q_locs), filtered_ecg(Q_locs), 'ko', 'DisplayName', 'Q wave');
plot(time(locs_R), filtered_ecg(locs_R), 'rv', 'DisplayName', 'R peak');
plot(time(S_locs), filtered_ecg(S_locs), 'mo', 'DisplayName', 'S wave');
plot(time(T_locs), filtered_ecg(T_locs), 'co', 'DisplayName', 'T wave');
title(sprintf('ECG with Detected PQRST Waves - Person %d', person_id));
xlabel('Time (s)');
ylabel('Voltage (mV)');
legend show;

% Display Results
fprintf('--- Person %d ---\n', person_id);
fprintf('PR interval:  %.1f ms\n', avg_PR);
fprintf('QRS duration: %.1f ms\n', avg_QRS);
fprintf('QT interval:  %.1f ms\n', avg_QT);
fprintf('RR interval:  %.1f ms\n', avg_RR);
fprintf('HR:          %.1f bpm\n', heart_rate);
```

```
    fprintf('QTc (Bazett): %.1f ms\n', QTc);
    fprintf('Heart Disease: %s\n\n', disease_flag);

 % Disease flag (label)
    flag = 0; % NO disease
    if PR*1000 < 120 || PR*1000 > 200 || ...
       QRS*1000 > 120 || ...
       QTc*1000 > 450 || ...
       HR < 60 || HR > 100
        flag = 1; % YES (abnormal)
    end

    % Store row of features
    features(end+1, :) = [PR*1000, QRS*1000, QT*1000, RR*1000, HR, QTc*1000, flag];
end
    % Create a table with column names
featureTable = array2table(features, ...
    'VariableNames', {'PR', 'QRS', 'QT', 'RR', 'HeartRate', 'QTc', 'DiseaseFlag'});

% Save to CSV for use in App
writetable(featureTable, 'ecg_features.csv');



%% Step 4: Machine Learning Model Training
% Split into training and testing (70/30)
cv = cvpartition(height(featureTable), 'HoldOut', 0.3);
TrainData = featureTable(training(cv), :);
TestData = featureTable(test(cv), :);

% Train classifier (Decision Tree here as example)
trainedModel = fitctree(TrainData(:,1:6),TrainData.DiseaseFlag);

%% Step 5: Training Accuracy
trainPred = predict(trainedModel, TrainData(:,1:6));
MLAccuracytrain = sum(trainPred == TrainData.DiseaseFlag) / numel(trainPred) * 100;
fprintf('Training Accuracy: %.2f%%\n', MLAccuracytrain);

figure
confusionchart(TrainData.DiseaseFlag, trainPred, ...
    'ColumnSummary', 'column-normalized', ...
    'RowSummary', 'row-normalized', ...
    'Title', 'Confusion Matrix - Training');
%% Step 6: Testing Accuracy
testPred = predict(trainedModel, TestData(:,1:6));
MLAccuracytest = sum(testPred == TestData.DiseaseFlag) / numel(testPred) * 100;
fprintf('Testing Accuracy: %.2f%%\n', MLAccuracytest);

figure
confusionchart(TestData.DiseaseFlag, testPred, ...
    'ColumnSummary', 'column-normalized', ...
```

```
    'RowSummary', 'row-normalized', ...
    'Title', 'Confusion Matrix - Testing');
```

```
function [trainedClassifier, validationAccuracy] = trainClassifier(trainingData)
% [trainedClassifier, validationAccuracy] = trainClassifier(trainingData)
% Returns a trained classifier and its accuracy. This code recreates the
% classification model trained in Classification Learner app. Use the
% generated code to automate training the same model with new data, or to
% learn how to programmatically train models.
%
%  Input:
%      trainingData: A table containing the same predictor and response
%       columns as those imported into the app.
%
%
%  Output:
%      trainedClassifier: A struct containing the trained classifier. The
%       struct contains various fields with information about the trained
%       classifier.
%
%      trainedClassifier.predictFcn: A function to make predictions on new
%       data.
%
%      validationAccuracy: A double representing the validation accuracy as
%       a percentage. In the app, the Models pane displays the validation
%       accuracy for each model.
%
% Use the code to train the model with new data. To retrain your
% classifier, call the function from the command line with your original
% data or new data as the input argument trainingData.
%
% For example, to retrain a classifier trained with the original data set
% T, enter:
%   [trainedClassifier, validationAccuracy] = trainClassifier(T)
%
% To make predictions with the returned 'trainedClassifier' on new data T2,
% use
%   [yfit,scores] = trainedClassifier.predictFcn(T2)
%
% T2 must be a table containing at least the same predictor columns as used
% during training. For details, enter:
%   trainedClassifier.HowToPredict


% Extract predictors and response
% This code processes the data into the right shape for training the model.
inputTable = trainingData;
predictorNames = {'PR', 'QRS', 'QT', 'RR', 'HeartRate'};
predictors = inputTable(:, predictorNames);
```

```
response = inputTable.DiseaseFlag;
isCategoricalPredictor = [false, false, false, false, false];
classNames = [0; 1];

% Train a classifier
% This code specifies all the classifier options and trains the classifier.
classificationTree = fitctree(...
    predictors, ...
    response, ...
    'SplitCriterion', 'gdi', ...
    'MaxNumSplits', 4, ...
    'Surrogate', 'off', ...
    'ClassNames', classNames);

% Create the result struct with predict function
predictorExtractionFcn = @(t) t(:, predictorNames);
treePredictFcn = @(x) predict(classificationTree, x);
trainedClassifier.predictFcn = @(x) treePredictFcn(predictorExtractionFcn(x));

% Add additional fields to the result struct
trainedClassifier.RequiredVariables = {'PR', 'QRS', 'QT', 'RR', 'HeartRate'};
trainedClassifier.ClassificationTree = classificationTree;
trainedClassifier.About = 'This struct is a trained model exported from Classification
Learner R2024b.';
trainedClassifier.HowToPredict = sprintf('To make predictions on a new table, T, use: \n
[yfit,scores] = c.predictFcn(T) \nreplace ''c'' with the name of the variable that is this
struct, e.g. ''trainedModel''. \n \nThe table, T, must contain the variables returned by: \n
c.RequiredVariables \nVariable formats (e.g. matrix/vector, datatype) must match the original
training data. \nAdditional variables are ignored. \n \nFor more information, see <a
href="matlab:helpview(fullfile(docroot, ''stats'', ''stats.map''),
''appclassification_exportmodeltoworkspace'')">How to predict using an exported model</a>.');

% Extract predictors and response
% This code processes the data into the right shape for training the model.
inputTable = trainingData;
predictorNames = {'PR', 'QRS', 'QT', 'RR', 'HeartRate'};
predictors = inputTable(:, predictorNames);
response = inputTable.DiseaseFlag;
isCategoricalPredictor = [false, false, false, false, false];
classNames = [0; 1];

% Perform cross-validation
partitionedModel = crossval(trainedClassifier.ClassificationTree, 'KFold', 5);

% Compute validation predictions
[validationPredictions, validationScores] = kfoldPredict(partitionedModel);

% Compute validation accuracy
validationAccuracy = 1 - kfoldLoss(partitionedModel, 'LossFun', 'ClassifError');
end
```